

Q. No	Understanding of Debugging Concepts (25)	Error Identification(25)	Root Causes Analysis (20)	Error Corrections and Solution Design(20)	Testing and Validation(10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						

	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

Debugging

Q1 Consider the following syntax-directed definition:

```
E → E1 + T    ( E.val = T.val )
E → T          { E.val = T.val }
T → num        { T.val = num.lexval }
```

For the input:

3 + 5

the compiler produces:

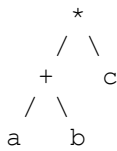
E.val = 5

Debug the given syntax-directed definition. Identify the incorrect semantic rule, explain its effect on attribute evaluation, provide the corrected rule, and determine the correct value of the expression.

Q2 . For the expression:

a + b * c

the compiler constructs the following syntax tree:



The compiler subsequently generates:

```
t1 = a + b
t2 = t1 * c
```

Debug the syntax-tree construction. Analyze the relationship between operator precedence and the generated tree, construct the corrected syntax tree, and generate the correct intermediate representation.

Q3. For the expression:

3 + 4 * 5

using:

```
E → E + T | T
T → T * F | F
F → num
```

the compiler annotates the root with:

```
E.val = 35
```

Analyze the annotated parse tree. Trace the synthesized attributes from the leaves to the root, identify the incorrect computation, and determine the correct value of the expression.

Q4 . The symbol table contains:

Identifier	Type
a	int
b	float
c	int
x	float

For the statement:

```
x = a + b * c;
```

the compiler reports:

```
Type Error: incompatible operands
```

Debug the type-checking process. Analyze each subexpression, determine the type of $b * c$ and $a + (b * c)$, identify where type conversion is required, and determine whether the assignment to x is valid.

Q5. Consider:

```
int a;
float b;
int x;
```

and the statements:

```
x = a + b;
b = a + x;
```

The type checker reports both statements as:

```
Valid
```

and does not generate any conversion.

Analyze the type-checking results. Determine the type of each expression, identify the required conversions, and explain how the type checker should process each assignment.

COURSE CODE : CSA1408

NAME : Ranjitha.R

COURSE NAME: Compiler Design

REG NO: 192421235

CO3 - AT 2 - Debugging Tool

Q1. Debugging the Syntax - Directed Definition

Given Syntax - Directed Definition ::

$$E \rightarrow E_1 + T \quad \{ E.val = T.val \}$$

$$E \rightarrow T \quad \{ E.val = T.val \}$$

$$T \rightarrow num \quad \{ T.val = num.lexval \}$$

For the input:

$$3 + 5$$

the compiler produces:

$$E.val = 5$$

1. Identify the error:

The incorrect semantic rule is:

$$E \rightarrow E_1 + T \quad \{ E.val = T.val \}$$

The rule calculates only the value of T. It completely ignores the value of E_1 . For addition, the value of the complete expression must be the sum of both operands.

2. Attribute evaluation:

For the input $3+5$:

$3 \rightarrow \text{num}$

$\text{num.laxVal} = 3$

Therefore:

$T.\text{Val} = 3$

$E_1.\text{Val} = 3$

using the incorrect rule:

$E.\text{Val} = T.\text{Val}$

$E.\text{Val} = 3$

For 5:

$5 \rightarrow \text{num}$

$\text{num.laxVal} = 5$

Therefore:

$T.\text{Val} = 5$

Therefore, the compiler incorrectly produces 5.

3. Effect of the incorrect rule:-

The incorrect rule causes the left operand to be lost during attributes evaluation.

Instead of:-

$$E.\text{Val} = E_1.\text{Val} + T.\text{Val};$$

it performs:-

$$E.\text{Val} = T.\text{Val}$$

Thus, the expression $3+5$ is incorrectly evaluated as 5.

4. Correct Semantic rule:

The corrected rule is:

$$E \rightarrow E_1 + T \quad \{ E.val = E_1.val + T.val \}$$

The other rules remain:

$$E \rightarrow T \quad \{ E.val = T.val \}$$

$$T \rightarrow num \quad \{ T.val = num.lexval \}$$

5. Correct evaluation:

$$E_1.val = 3$$

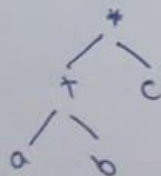
$$T.val = 5$$

$$\begin{aligned} E.val &= E_1 + T.val \\ &= 3 + 5 = 8 \end{aligned}$$

Q2. Debugging the Syntax Tree:

Given Expression:- $a + b * c$

Given syntax tree:



The compiler generates:

$$t_1 = a + b$$

$$t_2 = t_1 * c$$

1. Analyze the given tree:-

The given tree represents:

$$(a+b) * c$$

because $+$ is placed below $*$ in the tree. Therefore, the generated intermediate representation

$$t_1 = a + b$$

$$t_2 = t_1 * c$$

is consistent with inorder Tree

2. Operator precedence:-

In arithmetic expression:

$*$ has higher precedence than $+$

Therefore, in: $a + b * c$

The expression should be interpreted as:

$$a + (b * c)$$

3. Construct Syntax Tree:



4. Construct evaluation order:

First: $b * c$

Then: $a + (b * c)$

5. Correct intermediate representation:-

The Compiler should generate:

$$t_1 = b * c$$

$$t_2 = a + t_1$$

Q3 Analyze the Annotated parse Tree

Given Expression:-

$$3 + 4 * 5$$

Grammar:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow \text{num}$$

The Compiler gives : E.Val = 35

1. Identify operator precedence:

According to the grammar:

$$E \rightarrow E + T$$

+ is handled at T level, while

$$T \rightarrow T * F$$

* is handled at the E level.

Therefore,

* has higher precedence than +

The expression must be evaluated as:

$$3 + (4 * 5)$$

2. Evaluate the leaves:-

For 3 : $F.val = 3$

For 4 : $F.val = 4$

For 5 : $F.val = 5$

4. Evaluate addition:-

using: $E \rightarrow E + T$

We get:

$$\begin{aligned} E.val &= E_1.val + T.val \\ &= 3 + 20 \\ &= 23 \end{aligned}$$

5. Identify the incorrect computation:-

The compiler produces:

$$E.val = 35$$

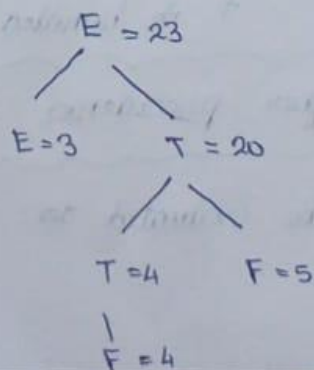
But,

$$35 = (3 + 4) * 5$$

This violates the precedence specified by the grammar.

Correct attribute

Flow:-



Q4. Debug the Type-checking Process

1. Analyze $b * c$

Types:

$b \rightarrow \text{float}$

$c \rightarrow \text{int}$

The multiplication has mixed operands.

$\text{float} * \text{int}$

The int value must be converted to float

Therefore:

$c: \text{int} \rightarrow \text{float}$

Then: $b * c$

$\text{float} * \text{float}$

Result:

$b * c \rightarrow \text{float}$

2. Analyze $a + (b * c)$

We have: $a \rightarrow \text{int}$

$b * c \rightarrow \text{float}$

Therefore:

$\text{int} + \text{float}$

The integer a must be converted to float

$a: \text{int} \rightarrow \text{float}$

Now: $\text{float} + \text{float}$

Therefore: $a + (b * c) \rightarrow \text{float}$

3. Analyze Assignment

The left size is : $x \rightarrow \text{float}$

The right size is : $a = b * c \rightarrow \text{float}$

Therefore, $\text{float} \rightarrow \text{float}$

This is valid

4. Required Conversions:

The compiler should perform:

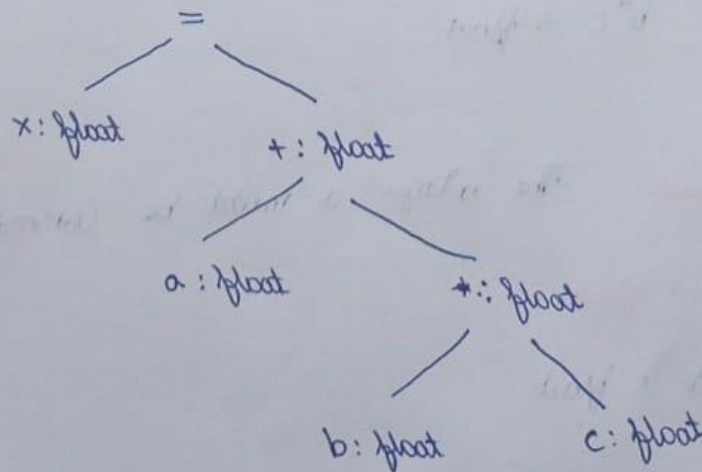
$c: \text{int} \rightarrow \text{float}$

$a: \text{int} \rightarrow \text{float}$

Conceptually :

$b * \text{float}(c)$ and $\text{float}(a) + (b * \text{float}(c))$

5. Convert type Tree:



Q6: Analyze Type Checking and Conversion.

Given:

int a;

float b;

int x;

Statement:

x = a + b;

b = a + x;

Statement 1: $x = a + b;$

Step 1: Determine operand types:

$a \rightarrow \text{int}$

$b \rightarrow \text{float}$

Therefore: $a + b$ have mixed types

$\text{int} + \text{float}$

The integer a must be converted to float.

$a: \text{int} \rightarrow \text{float}$

Therefore: $a + b \rightarrow \text{float}$

Step 2: Check assignment:-

The left side is: $x \rightarrow \text{int}$

The right side is: $a + b \rightarrow \text{float}$

So we have $\text{int} = \text{float}$

This requires a narrowing conversion:-

$\text{float} \rightarrow \text{int}$

This conversion can cause loss of the fractional part.
Conceptually:-

$$t_1 = \text{int_to_float}(a)$$

$$t_2 = t_1 + b$$

$$x = \text{float_to_int}(t_2)$$

Statement 2: $b = a + a$:

Step 1: Determine operand types:-

$$a \rightarrow \text{int}$$

$$x \rightarrow \text{int}$$

Therefore:

$$a + x \rightarrow \text{int}$$

Step 2: Check assignment:

The left side:

$$b \rightarrow \text{float}$$

The right side:

$$a + x \rightarrow \text{int}$$

Therefore:

$$\text{float} = \text{int}$$

The integer result must be converted to float.

$\text{int} \rightarrow \text{float}$

conceptually:

$$t_1 = a + x$$

$$b = \text{int_to_float}(t_1)$$

Comparison:

Statement	Expression Type	Required Conversion	Result.
$x = a + b$	float	int $\rightarrow$ float then float $\rightarrow$ int	valid, narrowing conversion
$b = a + x$	int	int $\rightarrow$ float	valid.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Debugging Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Error Identification	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Root Causes Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Error Correction & Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Testing and Validation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand